

Train Deep, Infer Shallow: Small Language Models for Semantic-ID Ads Retrieval

Xinyuan Wang¹, Arun Sethupat², Kushal Dave², Skanda Vasudevan², Dinesh Govindaraj², Akanksha Baid²

¹Arizona State University ²Pinterest

xwang735@asu.edu

{asethupat, kdave, skandavasudevan, dgovindaraj, abaid}@pinterest.com

Abstract

Early-stage ads retrieval must efficiently reduce a massive catalog into a high-recall candidate set under strict online latency constraints. Traditional embedding-based approaches rely on approximate nearest neighbor (ANN) search, utilizing fixed indices that lack dynamic, inference-time flexibility. In this paper, we study generative retrieval over Semantic IDs (SIDs) as a highly adaptable alternative. We represent each ad as a short, coarse-to-fine sequence of residual-quantized SIDs, utilizing a prefix trie to constrain decoding to valid catalog items. Crucially, this hierarchical structure unlocks an inference-time granularity knob: a model can “train deep, infer shallow” by decoding only the first few SID levels to efficiently retrieve coarse candidate buckets rather than individual items. To make this approach viable for production, we develop a deployable small language model (SLM) featuring self-supervised pre-training and deep-encoder/shallow-decoder architectures. Under fully cold-start evaluation on 3.1M input items, our autoregressive retriever outperforms a strong production ANN baseline. Furthermore, leveraging truncated decoding improves coarse recall by up to 34% while reducing latency by as much as 64%. Finally, we demonstrate that generative and ANN methods surface substantially different candidates; combining both routes recovers 1.45x more true positives than the stronger individual route alone, establishing generative retrieval with Semantic IDs as a powerful and complementary tool for web-scale candidate generation.

Keywords

Generative retrieval, Semantic IDs, Constrained decoding

1 Introduction

Modern recommendation and advertising systems use a multi-stage funnel: an early-stage retriever reduces a catalog of millions of items to a manageable candidate set, after which downstream rankers apply more expensive scoring [3, 16]. The stages have different objectives: ranking emphasizes top-of-list precision, whereas retrieval emphasizes recall, coverage, and latency. An effective retriever should therefore support different candidate granularities: coarse and broad when downstream ranking capacity is available, and finer when it is not.

Early-stage retrieval is commonly implemented with embedding-based retrieval (EBR), in which a two-tower model maps input items and ads into a shared space and approximate nearest-neighbor (ANN) search retrieves candidates [3, 10, 16, 20]. ANN is efficient, scalable, and difficult to beat. However, while ANN can vary the number of retrieved neighbors, it does not natively expose a hierarchical representation that lets the same model retrieve at different semantic granularities without rebuilding the index.

We instead formulate early-stage ads retrieval as generative retrieval over Semantic IDs (SIDs). A SID represents an item as a short sequence of coarse-to-fine discrete codes produced by residual quantization [24, 32, 38]; semantically related items therefore share code prefixes. Retrieval becomes generation of a SID sequence under a prefix trie, which guarantees that every decoded identifier is a valid, servable catalog item. Crucially, a length- k SID prefix represents a coarser item bucket, making decode depth an inference-time granularity knob: *train deep, infer shallow*. This differs from simply returning fewer ANN neighbors: it changes the semantic resolution of the retrieved candidates.

Our goal is to make SID-based generative retrieval practical for online serving. We study self-supervised pre-training, truncated decoding, SID geometry, and deep-encoder/shallow-decoder architectures for small language models (SLMs). On a fully cold-start, input-item-disjoint test set of 3.1M input items, these choices enable a 14-layer SLM to approach a 30-layer model’s retrieval quality while running at roughly half its decoding latency.

Contributions. All results use fully cold-start, input-item-disjoint evaluation, for which the ceiling from direct input-item memorization is zero. We make four contributions:

- We develop a **Semantic-ID generative retrieval framework** for early-stage ads retrieval, combining SLM training with trie-constrained decoding over valid catalog identifiers.
- We introduce **flexible truncated decoding**: a full-depth model can retrieve at any SID depth chosen at inference time. Truncation improves coarse recall by 9–34% while reducing autoregressive (AR) decoding latency by up to 64%.
- We identify a **deployable SLM recipe**: self-supervised pre-training, a deep encoder, and a shallow decoder. Pre-training improves HitRate@100 by 3–4% at unchanged serving latency, while a 14-layer model reaches HitRate@100=0.5028 at 0.103 s/batch.
- We show that **AR and ANN are complementary**: only 24% of their retrieved positives overlap, and their union recovers 1.45x the positives of the stronger individual retriever.

2 Related Work

Our work sits at the intersection of large-scale candidate retrieval, discrete item tokenization, and sequence modeling. We briefly review four threads: embedding-based approximate nearest-neighbor retrieval, generative retrieval with Semantic IDs, sequential recommendation, and autoregressive vs. non-autoregressive generation with constrained decoding.

2.1 Approximate Nearest-Neighbor Retrieval

Embedding-based retrieval (EBR) is the dominant approach to early-stage candidate generation: two-tower models map input items and

¹*Work performed while the primary author was an intern at Pinterest.

ads into a shared vector space, and approximate nearest-neighbor (ANN) search retrieves candidates [3, 16, 17, 23, 44]. Product quantization, graph indexes such as HNSW, and systems such as FAISS make ANN practical at web scale [12, 18, 20, 28]. We use a strong production two-tower ANN system as our principal baseline and study when generative retrieval complements its fixed-index operating point.

2.2 Generative Retrieval and Semantic IDs

Generative retrieval recasts retrieval as sequence generation: a model *generates* the identifier of the target item token by token [14, 19, 26]. DSI and NCI [37, 40] first showed that a Transformer can memorize a corpus and emit document IDs, while GENRE used a prefix trie to constrain entity generation to valid identifiers [4]. Subsequent work scales and refines this paradigm for IR and recommendation, e.g., RIPOR, GLEN, LTRGR, and Self-Retrieval [25, 27, 36, 46]. A central design question is how to tokenize items: Semantic IDs address this by residual-quantizing an item’s content embedding into a short sequence of discrete codes so that similar items share code prefixes [24, 38]. TIGER [32] brought Semantic IDs to recommendation, and follow-up work integrates Semantic IDs with LMs and industrial-scale systems [5, 8, 29, 33, 34, 41, 42, 47]. RPG [15] generates long Semantic IDs in parallel, and several papers argue that generative and dense retrieval are complementary [43, 45], motivating hybrid systems.

2.3 Sequential Recommendation

Sequential recommenders predict a user’s next item from interaction history. GRU4Rec uses recurrent networks for session-based recommendation [13], SASRec uses causal self-attention [22], and BERT4Rec uses bidirectional attention with a masked-item objective [35]. We use GRU4Rec and SASRec as learned baselines, and adapt BERT4Rec-style masked-item pre-training for NAR and T5-style span corruption [31] for AR Semantic-ID generators.

2.4 Autoregressive vs. Non-Autoregressive Generation and Constraints

Our models use standard Transformer sequence-generation architectures [39]. We compare T5-style encoder-decoder generation [31] with decoder-only and prefix-LM variants [7, 30]. As a faster alternative, non-autoregressive (NAR) models predict all SID positions in parallel [9, 11]; we adapt this approach to per-level Semantic-ID prediction. Because unconstrained generation can produce invalid identifiers, all generative models decode through a catalog trie, following constrained generation methods such as GENRE [4]. To our knowledge, we are the first to combine Semantic-ID generative retrieval with depth-flexible truncation and an explicit AR/NAR/ANN comparison for early-stage ads retrieval under cold-start evaluation.

3 Preliminaries

3.1 Semantic IDs and the RQ-VAE Hierarchy

A *Semantic ID* (SID) represents an item by a short, ordered sequence of L discrete codes $\mathbf{s} = (c_1, \dots, c_L)$, with each $c_\ell \in \{1, \dots, V\}$ drawn from a per-level codebook of size V . The sequence is produced

by a residual-quantized autoencoder (RQ-VAE) [24, 38] over the item’s content embedding $\mathbf{z}$. For our work, we derive Semantic IDs using OmniSage [1] and PinCLIP [2] embeddings. Starting from residual $\mathbf{r}_1 = \mathbf{z}$, each level chooses the nearest codeword from its own codebook $\{\mathbf{e}_j^{(\ell)}\}_{j=1}^V$ and passes the leftover residual down:

$$c_\ell = \arg \min_{j \in \{1, \dots, V\}} \|\mathbf{r}_\ell - \mathbf{e}_j^{(\ell)}\|_2, \quad \mathbf{r}_{\ell+1} = \mathbf{r}_\ell - \mathbf{e}_{c_\ell}^{(\ell)}, \quad \ell = 1, \dots, L, \quad (1)$$

so the SID is $\mathbf{s} = (c_1, \dots, c_L)$. Quantization is *hierarchical*: later code levels encode the residual information left by earlier levels [24, 32]. We rely on two properties throughout. (i) Levels are strongly dependent, coarse-to-fine: c_ℓ is only meaningful given $c_{<\ell}$. As established in prior recommendation and image generation literature [24, 32, 38], residual quantization inherently forces semantically related items to share code prefixes. Therefore, items that share a length- k prefix $s_{\leq k} = (c_1, \dots, c_k)$ are semantically close. For a length- k prefix $\mathbf{p}$, let

$$B(\mathbf{p}) = \{i \in \mathcal{I} : \mathbf{s}_{\leq k}^{(i)} = \mathbf{p}\} \quad (2)$$

be the set of catalog items ($\mathcal{I}$ the catalog) whose first k codes equal $\mathbf{p}$. Shorter prefixes name coarser, larger buckets; the average *fan-out* $|B(\mathbf{p})|$ shrinks monotonically with depth and is close to 1 at $k=L$ (a full SID typically names one or a small handful of items).

We study two SID shapes, denoted $L \times b$ for L levels of $V = 2^b$ codes each: 5×11 (5 levels, 2048 codes/level; the original codebook) and 4×8 (4 levels, 256 codes/level; a shallow, coarse-retrieval-native shape). Their ad catalogs hold 29.6M and 13.1M unique SIDs respectively. Because V differs across shapes, the same level index denotes *different* granularities—a caveat we return to in §5.5.

Fan-out and populated buckets. Because levels are coarse-to-fine, the number of *populated* k -prefix buckets (distinct length- k prefixes in the catalog) grows far more slowly than the theoretical V^k (Fig. 1); the residual quantizer concentrates items into a relatively small set of coarse buckets. Equivalently, the per-prefix fan-out $|B(\mathbf{p})|$ collapses with depth (Table 1): shallow prefixes already name one of only tens-to-thousands of real coarse buckets. This structure is exactly what the inference-time depth knob (§4.5, §5.4) exploits.

Table 1: Average #distinct full SIDs under each prefix- k .

SID	prefix-1	prefix-2	prefix-3	prefix-4	prefix-5
5×11	14.5k	34.3	2.69	1.23	1.00
4×8	51.5k	233.7	5.51	1.00	–

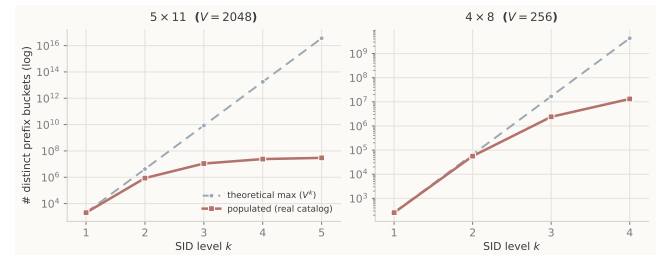


Figure 1: Populated prefix buckets per SID level vs. the theoretical max V^k (log scale).

3.2 Retrieval as Sequence Generation

Each retrieval example is centered on an *input item*, which serves as the anchor for candidate ad retrieval. An input is triggered when a user clicks an item. In addition to the input item, we use up to N co-engaged related items as contextual signals, and the retrieval targets are the ads associated with the input context through the impression-joined item–ad co-engagement data. We represent the input item, related items, and target ads in the same SID space. Formally,

$$x = (s^{\text{input}}, s_1^{\text{rel}}, \dots, s_N^{\text{rel}}), \quad \mathcal{Y} = \{y_1, \dots, y_M\},$$

where s^{input} is the SID of the input item, s_i^{rel} is the SID of the i -th related item, and y_j is the SID of a target ad. Each SID contains L code levels. Given the input context x , the task is to return a ranked top- K list of target-ad SIDs.

We formulate the task as *Semantic-ID sequence generation* and use *trie-constrained decoding* to ensure that retrieved SIDs correspond to valid catalog items. The multi-target formulation reflects early-stage retrieval, where multiple ads may be relevant to the same input context. We consider both autoregressive (AR) and non-autoregressive (NAR) formulations, detailed in §4.1.

3.3 Trie-Constrained and Truncated Beam Search

Free generation can produce code sequences that do not correspond to real catalog items. We therefore compile valid SIDs into a prefix trie and mask each beam-search step to the legal children of the current prefix. Every completed sequence is consequently a valid, servable item (all completed sequences are valid catalog SIDs).

The trie also enables *truncated* decoding. Rather than decode all L levels, we can stop at depth k and return the top- K distinct length- k prefixes, each representing a coarse item bucket. This provides an inference-time granularity knob: smaller k retrieves broader buckets, while full-depth decoding retrieves near-singleton items. As shown in §5.4, re-decoding to depth k covers more distinct coarse buckets than decoding to depth L and then taking prefixes, improving coarse recall while reducing latency.

3.4 Cold-Start Evaluation Setup

All experiments use a fully input-item-disjoint cold-start split: none of the 3.12M evaluation input items appear in the 27.5M training rows (zero input-item overlap), while every ground-truth SID remains reachable through the trie. Models must therefore generalize to unseen input items rather than memorize input-specific patterns. In this setting, the relevant reference is the popularity floor (HitRate@10=0.0028), not a recall ceiling of 1.0; the best model reaches HitRate@10=0.204, or 73× the floor.

4 Method

We model early-stage ads retrieval as sequence generation over Semantic IDs (§3). Given an input SID and its related SIDs, an encoder–decoder or single-stack Transformer produces per-level logits over the SID vocabulary, and trie-constrained beam search returns the top- K valid identifiers. We now formalize the model space, data, training objectives, and the truncated decoding strategy that makes the approach deployable.

4.1 Model Space

Let x denote the input SIDs (input + related) and $\mathbf{s} = (c_1, \dots, c_L)$ a candidate SID. As in §3.2, retrieval is *sequence generation*: a model scores $\mathbf{s}$ by $p_\theta(\mathbf{s} | x)$ and we take the trie-constrained top- K :

$$\hat{\mathcal{Y}} = \underset{\mathbf{s} \in \mathcal{T}}{\text{top-}K} p_\theta(\mathbf{s} | x), \quad (3)$$

where $\mathcal{T}$ is the set of valid catalog SIDs (the trie).

Our reference model is an *encoder–decoder autoregressive (AR)* generator: a T5-style [31] bidirectional encoder over (input+related) with an autoregressive decoder that emits $c_1, \dots, c_L$ under cross-attention,

$$(\text{AR}) \quad p_\theta(\mathbf{s} | x) = \prod_{\ell=1}^L p_\theta(c_\ell | x, c_{<\ell}). \quad (4)$$

This respects the coarse-to-fine dependence of the SID hierarchy (§3.1) at the cost of L sequential decode steps.

For contrast we also study a *non-autoregressive (NAR)* generator, which predicts all levels in parallel with per-level classification heads:

$$(\text{NAR}) \quad p_\theta(\mathbf{s} | x) = \prod_{\ell=1}^L p_\theta(c_\ell | x). \quad (5)$$

The trie restores a joint constraint at decode time, but the per-level scores remain conditionally independent, which limits attainable fine-level accuracy.

We also compare single-stack AR backbones—GPT-style decoder-only and UniLM-style prefix-LM [7, 30]—with encoder–decoder AR. All models share the same SID space, data, and trie-constrained decoding, so differences isolate the effect of architecture and factorization.

4.2 Data Construction

We construct each example from impression-joined item–ad co-engagement: an input-item SID, up to N related-item SIDs as context, and a set of target-ad SIDs $\mathcal{Y}$. All items are tokenized using the SID codebook of the chosen shape (§3.1). This multi-target formulation matches early-stage retrieval, where multiple ads may be relevant to one input item. We use the same construction for both SID shapes, cap each input item’s target set at T , and enforce input-item-disjoint train–test splits (§3.4).

4.3 Supervised Fine-tuning

We fine-tune models to generate the target SID set from the input context. The supervised objective is the sum of per-level cross-entropies over target SIDs:

$$\mathcal{L}_{\text{FT}}(\theta) = - \sum_{\mathbf{y} \in \mathcal{Y}} \sum_{\ell=1}^L \log p_\theta(y_\ell | x, y_{<\ell}). \quad (6)$$

For AR, $y_{<\ell}$ denotes teacher-forced preceding SID codes; for NAR, this conditioning is omitted and levels are predicted independently. We select checkpoints by validation HitRate@10. Pre-training changes only the fine-tuning initialization; the resulting model is otherwise a standalone retriever.

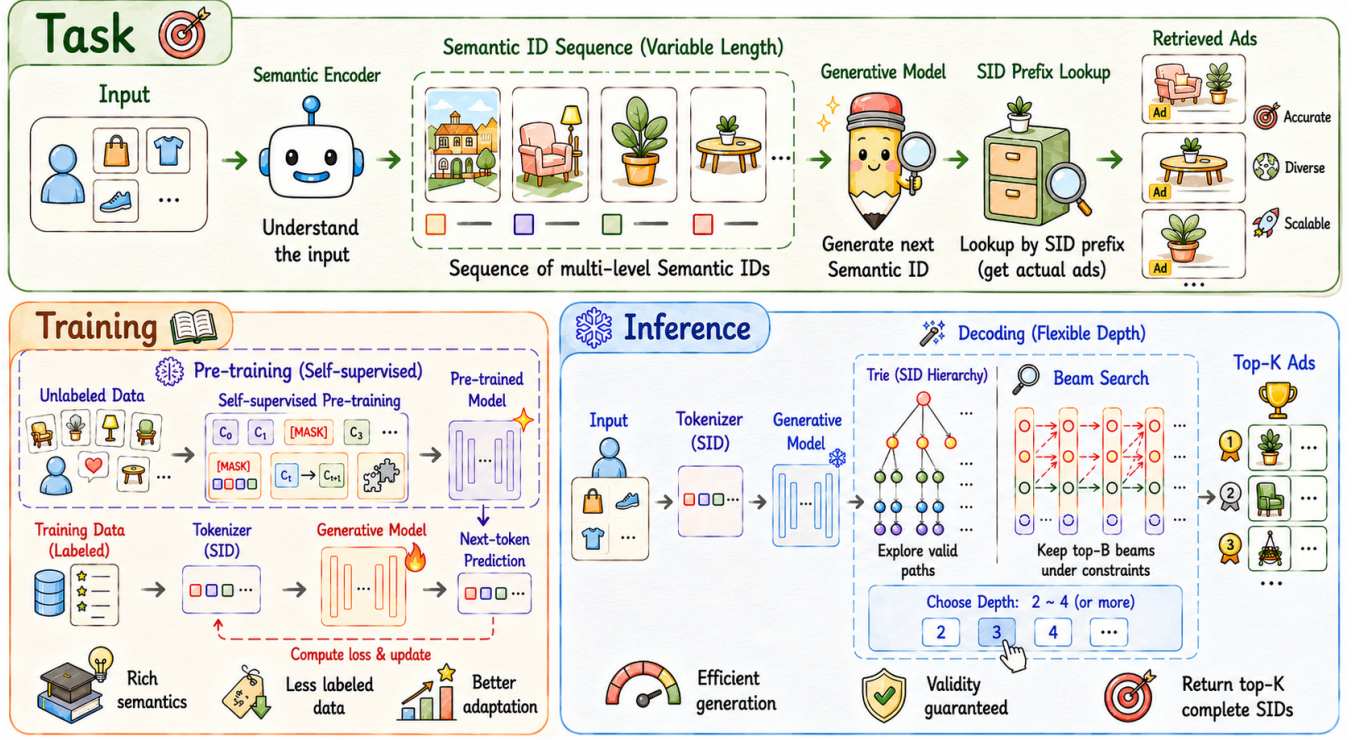


Figure 2: System overview. A bidirectional encoder reads the input item and its related Semantic IDs, and a decoder generates target-ad SIDs under a prefix-trie constraint. Beam search returns valid SIDs, which are mapped to servable ads by SID-prefix lookup. Models are self-supervised pre-trained and then fine-tuned. At inference, decoding can stop at any depth k , providing a granularity dial from broad coarse buckets to near-singleton items.

4.4 Self-Supervised Pre-training

To improve small models without changing the serving path, we add a self-supervised pre-training stage on the same co-engagement rows, mirroring the pre-train-then-finetune recipe of language models [6, 31]. For each row we deterministically partition the deduplicated SID multiset $\text{dedup}([\text{input}] \cup \text{related} \cup \text{targets})$ into a context x_{ctx} and a held-out subset $\mathcal{M}$ via a per-row hash. The held-out items are then treated as self-supervised targets under the *same* generation loss as in Eq. 6, with x replaced by x_{ctx} and $\mathcal{Y}$ by $\mathcal{M}$.

For AR, this realizes a T5-style span-corruption objective (mask contiguous spans and generate them) [21, 31]; for NAR, it becomes a BERT4Rec-style masked-item objective (predict held-out items from a bidirectional context) [35]. Fine-tuning then proceeds exactly as in §4.3, but warm-starting from the pre-trained weights. Architecture, learning-rate schedule, data, and supervised step budget are identical with and without pre-training, so the observed gains can be attributed to initialization (§5.6).

4.5 Inference: Trie-Constrained and Truncated Decoding

Inference-time control over decode depth is one of the main differentiators of our approach; §5.4 quantifies its effect.

Trie-constrained decoding. Given the per-level logits (an $L \times V$ matrix), we realize the top-K in Eq. 3 using beam search masked by the catalog trie (§3.3). Let $\mathcal{A}(c_{<\ell})$ be the set of codes that extend prefix $c_{<\ell}$ to a legal catalog prefix; we only consider $c_\ell \in \mathcal{A}(c_{<\ell})$ and score partial SIDs by their accumulated log-probability:

$$c_\ell \in \mathcal{A}(c_{<\ell}), \quad \text{score}(s_{\leq \ell}) = \sum_{j=1}^{\ell} \log p_{\theta}(c_j | x, c_{<j}). \quad (7)$$

At each level we retain the top-B beams by this score. Because every surviving path stays on the trie, every completed sequence is a valid catalog SID. This constraint is essential for NAR (whose unconstrained outputs are rarely valid) and still beneficial for AR (§5.8).

Truncated decoding (train deep, infer shallow). Crucially, we rarely need to decode all L levels for early-stage retrieval: matching on the first 2–3 SID levels already identifies coarse buckets $B(\mathbf{p})$ (§3.1). Because the trie is a prefix structure, we can therefore decode only to depth $k \leq L$ by running the same trie-constrained beam for k levels and stopping, returning the top-K distinct length- k prefixes:

$$\hat{\mathcal{Y}}_k = \text{top-K } p_{\theta}(\mathbf{p} | x), \quad \mathbf{p} \text{ naming a bucket } B(\mathbf{p}) \text{ (Eq. 2),} \quad (8)$$

where $\mathcal{T}_{\leq k}$ is the set of legal k -prefixes. Setting $k=L$ recovers full-depth retrieval; smaller k yields coarser, wider buckets. Thus a

single trained model acts as both a coarse retriever (small k) and a near-item-level retriever (large k), and k becomes an inference-time granularity dial: *train deep, infer shallow*. Importantly, truncated decoding is not only cheaper but also more accurate at coarse levels than decoding to full depth and then taking prefixes. A full-depth beam tends to waste budget on paths that only diverge late, so the top- K sequences share few distinct short prefixes. Truncating to depth k shrinks the search tree from L levels to k levels (an orders-of-magnitude reduction in reachable leaves given the fan-out collapse in Table 1), so the same beam width covers more distinct buckets and improves per-level recall while reducing AR latency (§5.4). By contrast, simply widening the beam at full depth barely moves recall but increases latency, so the effective lever for early-stage retrieval is *going shallower*, not *searching wider*.

5 Experiments

We demonstrate the efficacy of our approach through the following sequence of questions:

- **(RQ1) Retrieval effectiveness:** Does Semantic-ID generative retrieval outperform a production ANN and other baselines in cold-start? (§5.2)
- **(RQ2) Representation quality and decoding granularity:** How is representation quality distributed across Semantic ID (SID) levels, and can *truncated re-decoding* trade fine-grained resolution for faster, high-recall retrieval? (§5.3,5.4)
- **(RQ3) Deployment trade-offs:** Motivated by cross-level redundancy, can a self-supervised SLM with a shallow 4×8 codebook and deep-encoder/shallow-decoder architecture match large-model performance within online latency limits, and which design choices matter most? (§5.5,5.6,5.7,5.8)
- **(RQ4) Head vs. Tail comparison:** Where do Semantic-ID and ANN search diverge across varying SID depths and head, torso, or tail population and what retrieval performance does combining them unlock? (§5.9,5.10)

Why two SID codebooks? We use a 5×11 codebook as the comparison substrate because the production ANN index is defined in this space; all direct ANN comparisons in RQ1 and RQ4 and the per-level diagnosis in RQ2 use 5×11 . A shallower 4×8 codebook is then introduced for deployment: motivated by RQ2’s finding that useful signal concentrates in shallow SID levels, it supports the online-feasibility study in RQ3 by applying the 5×11 diagnosis to a deployable design rather than serving as a separate benchmark.

5.1 Experimental Setup

Data: We use impression-joined item–ad co-engagement (§4.2), tokenized into Semantic IDs. Models are trained on 29.6M rows; the test set is the *full* 3,120,838 input items with no subsampling. We report on two SID shapes (§3): the original 5×11 (5 levels, 2048 codes/level) and the migration target 4×8 (4 levels, 256 codes).

Table 2: Dataset statistics. Evaluation is fully cold-start.

Test input items (no subsampling)	3,120,838
#unique ad SIDs, 5×11	29,642,770
#unique ad SIDs, 4×8	13,138,295
Trie reachability ceiling	1.000

Cold-start: Evaluation is fully input-disjoint from training (§3.4), so the ceiling from direct input-item memorization is 0. We therefore compare against the popularity baseline as a trivial non-personalized reference. Performance above this baseline measures generalization to unseen input items rather than direct input memorization.

Metrics: We use a single evaluation pipeline across all retrieval routes. Our primary metric is **HitRate@K** ($K \in \{10, 100\}$), which counts an input as recalled if at least one ground-truth ad/SID appears in the top- K results. We read recall at three granularities: full-SID HitRate@K, prefix-level HitRate@K over length- ℓ prefixes, and SetRecall@|C| against a fixed ground-truth denominator to compare decode depth (|C| denotes the number of ad candidates after prefix expansion). To account for multi-ad targets, we report **SetRecall@100** (fraction of targets recovered). Ranking quality is **NDCG@10** and **MRR@10**, and **Precision@10**. Catalog-level health is **Gini@100**. We record decode latency (s/batch) and confirm valid outputs under trie constraints.

Prefix notation (*prefix* vs. *trun-prefix*). Two distinct depth operations recur throughout the experiments, and we name both after the length- ℓ code *prefix* they return, distinguishing only the operation. A **prefix- ℓ** is a *per-level read-off* of a single *full-depth* decode: we decode all L levels and score the length- ℓ prefix (“take-prefix”). A **trun-prefix- k** is a *truncated re-decode*: the beam runs over only the first k levels, competing directly over length- k prefixes. Both return a coarse code prefix; the trun-prefix is the *truncated* variant (marked “trun”), the prefix the full-depth read. Since they are not the same operation, we always mark the truncated one. This prefix / trun-prefix convention holds in every table and figure below.

Configurations. All models are trained with PyTorch DDP on **8×NVIDIA A100-40GB** (bf16 AMP, TF32, torch.compile) and evaluated with a **48-way sharded** decode pipeline. All models are implemented in PyTorch 2.6 (CUDA 12.4) with native nn.Transformer modules; the decoder-only baseline additionally uses HuggingFace transformers. Generative models decode under a trie constraint; ANN retrieves top-100 within the legal catalog.

Parameter settings: The generative model AR is a T5-style encoder-decoder (12 encoder + 12 decoder layers, $d_{\text{model}}=768$, 12 heads, $d_{\text{ff}}=3072$, 214M params). Inputs use up to $N=15$ related items; supervision uses up to T target SIDs ($T=8$ for 5×11 , 24 for 4×8 , matching each shape’s per-input target-count distribution). We train ~ 2 epochs ($\sim 422k$ steps at effective batch 1024) with 10% linear warmup and AdamW, and select the checkpoint by inline HitRate@10. Decoding uses top- $K=100$ with beam width $B=100$; unless noted, truncate is off (we decode at full depth). Per-shape deltas are the SID dimensions and T above; the deep-encoder/shallow-decoder points vary only the encoder/decoder depth. We denote encoder and decoder depth as $E + D$; for example, 12+12 denotes a 12-layer encoder and a 12-layer decoder.

Baselines: (i) Non-generative baselines: heuristic (*random walk*, *BM25*, *random-walk-item*, *exact prefix match*) and a production two-tower ANN; we use the same PinCLIP representation z (§3.1) for ANN retrieval; and (ii) generative baselines: AR (decoder-only GPT, prefix-LM) and NAR (four depths). All methods are trained and evaluated on the same train/test split using the same SID representation where applicable, providing a consistent cold-start comparison.

Table 3: Cold-start retrieval benchmark on the full 3.12M-input test set (5×11 , trie-constrained decoding).

Model	Route	Params	HitRate@10	HitRate@100	NDCG@10	MRR@10	SetRecall@100	Gini@100↓
<i>Non-generative baselines (heuristic + production ANN)</i>								
Random Walk	—	—	0.0064	0.0215	0.0035	0.0026	0.0067	—
BM25	—	—	0.0314	0.0561	0.0234	0.0210	0.0142	0.4405
Random Walk Item [§]	—	—	0.0339	0.0747	0.0192	0.0148	0.0282	—
Exact Prefix Match	—	—	0.0412	0.1220	0.0226	0.0170	0.0511	—
ANN (two-tower)	ANN	—	0.1923	0.3767	0.1194	0.0970	0.1519	0.615
<i>Generative retrieval baselines</i>								
NAR (6)	NAR	23M	0.0672	0.1966	0.0380	0.0291	0.0818	0.832
NAR (8)	NAR	53M	0.0781	0.2147	0.0451	0.0350	0.0912	0.829
NAR (12)	NAR	101M	0.0845	0.2249	0.0493	0.0386	0.0967	0.826
NAR (24)	NAR	324M	0.0928	0.2376	0.0547	0.0430	0.1036	0.823
AR (Decoder-only)	AR-1stk	36M	0.1500	0.3459	0.0869	0.0676	0.1717	0.910
AR (Prefix-LM)	AR-1stk	36M	0.1476	0.3464	0.0840	0.0646	0.1724	0.908
AR (12+12)	AR-encdec	214M	0.2040	0.4223	0.1224	0.0974	0.2266	0.898
<i>Deep encoder shallow decoder model</i>								
AR 24+6[†]	AR-encdec	424M	0.2097	0.4228	0.1262	0.1006	0.2284	0.894

[†] The 24+6 model ($d=1024$) was trained under the 8-GPU sweep protocol. Its same-protocol 12+12 reference achieves HitRate@100=0.4168; 24+6 improves on this reference by 1.4 points. [§] Random Walk Item is a cold-start *negative control*; its low recall corroborates a near-zero memorization ceiling.

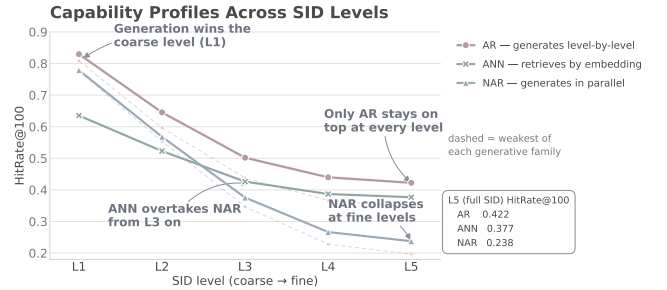
5.2 Generative Retrieval vs. ANN (RQ1)

We begin on the 5×11 codebook—the space of the production ANN index—so the ANN comparison is like-for-like. Table 3 evaluates all paradigms on this common ruler. The quality ordering is **enc-dec AR > ANN > single-stack AR > NAR**: the AR model reaches HitRate@100=0.422, a +12% gain over ANN and +78% over the best NAR. Two points stand out: (i) the entire NAR family sits below ANN—even the 324M 24-layer NAR attains only 63% of ANN—so NAR is the *fast* generative option, not the quality leader; (ii) ANN is a strong baseline: it matches the best generative model on ranking (NDCG@10 0.119 vs. 0.122; MRR@10 0.097 vs. 0.097), has the healthiest distribution (Gini 0.615), and the lowest (offline) cost; only enc-dec AR surpasses it.

5.3 Quality Distribution Over SID Levels (RQ2)

We next ask *where* models are strong along the SID hierarchy, using per-level HitRate@100 at SID depth ℓ , which counts a hit if the first ℓ codes match, under the same ground truth as full-depth recall (Fig. 3, Table 4). The three routes have distinct *capability shapes*: **NAR** is strong at coarse levels ($L1 \approx 0.78$) but collapses toward the leaves ($L5 \approx 0.24$); **ANN** is weaker at coarse levels ($L1 \approx 0.64$) but has the flattest curve, overtakes NAR around $L3$, and finishes

$1.6\times$ higher at $L5$; **encoder-decoder AR** is highest at all levels ($L1 \approx 0.83$, $L5 \approx 0.42$), combining generative coarse-level strength with strong fine-level recall. The NAR/ANN crossing between $L2$ – $L3$ underpins the AR–ANN complementarity in §5.9 and motivates our decoding and codebook design: for early-stage retrieval, most useful signal lies in the first few SID levels, so decoding all the way to leaves is often wasteful when matching at levels 2–3 can suffice and even improve recall at fixed cost (§5.4, §5.5).

**Figure 3: ANN, NAR, and AR HitRate@100 across SID levels.****Table 4: Here ℓ is a match depth on one full-depth decode; both metrics are non-increasing in ℓ by construction.**

Metric	Model	prefix-1	prefix-2	prefix-3	prefix-4	prefix-5
HitRate@100	ANN	0.635	0.523	0.426	0.387	0.377
	NAR	0.779	0.568	0.376	0.267	0.238
	AR	0.830	0.645	0.502	0.440	0.422
Precision@10	ANN	0.402	0.128	0.045	0.026	0.021
	NAR	0.538	0.190	0.061	0.022	0.011
	AR	0.420	0.112	0.044	0.030	0.026

5.4 Flexible Truncated Decoding (RQ2)

This is our central mechanism and the capability ANN cannot offer. Because SIDs are hierarchical, a *single* full-depth model can be evaluated at any prefix depth k at deployment time, with *zero* re-training—one full-depth model is a coarse retriever at every k .

(a) Trun-prefix beats prefix on recall. At cut depth k , re-decoding to a *trun-prefix- k* (beam over length- k prefixes) improves HitRate@100 over a *prefix- k* read for every (model, k), with the largest gains at shallow k (Table 5: NAR $k=2$ +0.170, AR $k=2$ +0.120). Precision@10 moves the other way, especially at $k=2$ (NAR −0.106, AR −0.022)

and nearly vanishes by $k=4$, because trun-prefix allocates its budget to *distinct* coarse buckets while prefix- k repeats a few high-probability prefixes. For early-stage retrieval, this trade—slightly lower precision for many more distinct coarse candidates—is ideal, since a downstream ranker re-scores items within each bucket.

Table 5: Truncated re-decode (trun-prefix- k) vs. full-depth take-prefix (prefix- k) at cut depth k (5×11 AR/NAR)

Model	k	HitRate@100			Precision@10		
		trun-pfx	prefix	Δ	trun-pfx	prefix	Δ
NAR	2	0.734	0.564	+0.170	0.081	0.187	-0.106
	3	0.453	0.366	+0.087	0.029	0.059	-0.029
	4	0.282	0.254	+0.027	0.013	0.020	-0.007
AR	2	0.766	0.645	+0.120	0.090	0.112	-0.022
	3	0.566	0.501	+0.065	0.044	0.044	+0.000
	4	0.458	0.439	+0.019	0.030	0.030	-0.000

(b) Why re-decode is both faster and more accurate. The mechanism is a mismatch between where a full-depth beam *spends* its width and where the useful retrieval signal *lives*. A full-depth beam ranks length- L paths, so it preferentially explores paths that only diverge at the deep levels—exactly the levels whose fan-out has already collapsed to ≈ 1 (Table 1: 5×11 L4→L5 barely splits). It therefore burns budget separating near-identical deep endings that a downstream ranker never needed distinguished, and its top-100 collapses onto only ~ 63 distinct L3 prefixes (37% wasted). Truncating to depth k re-spends that same width on *distinct* coarse buckets: the search frontier shrinks from an L -deep tree to a k -deep one (orders of magnitude smaller by the fan-out collapse), so the beam covers more real buckets—higher coarse recall—over fewer levels. In short, truncation converts wasted deep-level resolution into coarse-level coverage. The speed dividend follows for free: AR latency drops monotonically with k (full 0.498 $\rightarrow$ $k=2$ 0.185 s/b, -64%); NAR latency is flat (its bottleneck is the CPU trie-beam, not the forward pass), so the speed win is AR’s.

(c) Beam width saturates, while shallower decoding is the stronger lever. Table 6 shows that increasing beam width helps only when search is under-provisioned. For NAR, increasing the beam from 50 to 100 raises full-depth HitRate@100 from 0.1685 to 0.2247, whereas increasing it further to 200 yields only a small gain. AR is already saturated at beam 50, with HitRate@100 remaining nearly unchanged across beam widths, while doubling the beam from 100 to 200 increases latency by 117%. In contrast, reducing decode depth directly changes the granularity of the search. At $k=2$, truncated re-decode improves HitRate@100 by +0.120 for AR and +0.170 for NAR over taking the same prefix from a full-depth beam (Table 5). Thus, beam width mainly controls search capacity,

Table 6: Beam-width control under full-depth decoding.

Beam Width	NAR HitRate@100	AR HitRate@100
50	0.1685	0.4223
100	0.2247	0.4208
200	0.2310	0.4238

whereas decode depth controls what the hypotheses compete over. Once the beam is sufficiently wide, shallower decoding provides the more effective lever for coarse retrieval.

5.5 Is a Shallow Codebook better? (RQ3)

Everything above ran on 5×11 and converged on one insight—shallow is enough. We now act on it: if the useful signal lives in the first few levels, the codebook itself should be shallow. This is why a second SID shape (4×8) enters, and it is derived from the 5×11 analysis, not chosen in parallel. From here on, the question is online feasibility—matching the large model’s quality at a deployable cost. The truncation insight (§5.4) says the useful retrieval signal is concentrated in the shallow levels. We test this at the source—the codebook geometry—by re-quantizing the catalog into a natively shallow shape, 4×8 (4 levels), and re-running the paradigm study on it. If shallow is enough, a shape that packs the useful granularity into fewer levels should retrieve as well as the deeper 5×11 while decoding cheaper.

Why 4×8 : the coarse→fine gradient fits in four levels. Table 1 shows that 4×8 reaches uniqueness cleanly at L4 (fan-out 1.00) while preserving a useful mid-level split at L3 (5.51), whereas 5×11 adds little at the final step (L4→L5 fan-out 1.23 $\rightarrow$ 1.00), making its deepest level nearly dead weight for retrieval. Although the smaller codebook needs slightly more codes to reach the same granularity, it places the full gradient where the signal appears to lie, at roughly half the catalog size (13.1M vs. 29.6M SIDs).

Table 7: HitRate@100 on 5×11 vs. 4×8 .

SID	Model	HitRate@100
5×11	NAR(12 layer)	0.2249
	AR (12+12)	0.4223
	AR (24+6)	0.4228
4×8	NAR(12 layer)	0.3215
	AR (12+12)	0.5024
	AR (24+6)	0.5034

Base results replicate the paradigm study and expose a deep-encoder/shallow-decoder operating point on both shapes.

Table 7 shows **AR** $\gg$ **NAR** on both shapes (4×8 : 0.502 vs 0.322, +56%; 5×11 : 0.422 vs 0.225, +88%), so the paradigm ranking is shape-invariant. The revealing point is the deep-encoder/shallow-decoder model (a 24-layer encoder with a shallow decoder), which *matches* encoder-decoder AR on both shapes (4×8 24+4: 0.503 vs 0.502; 5×11 24+6: 0.423 vs 0.422) while decoding faster—on 4×8 the 4-layer decoder is 1.8 $\times$ faster than 12+12 (48-way fast-eval), and on 5×11 the same-protocol 24+6 is 22% faster than the fair 12+12 base (0.381 vs 0.491 s/b). A deep encoder plus a shallow decoder thus suffices regardless of codebook depth, motivating the deployable operating point of §5.7.

Table 8: Cross-SID comparison via encoder-decoder AR.

SID	L1	L2	L3	L4	L5 (=HitRate@100)
5×11	0.830	0.645	0.502	0.440	0.422
4×8	0.878	0.747	0.603	0.502	–

The granularity trap: SID-level recall is not comparable across shapes. Table 1 shows that the same level represents different granularities: an L2 bucket holds 234 SIDs at 4×8 but only 34 at 5×11 , so the deepest-level comparison in Table 8 (4×8 0.502 > 5×11 0.422) is a **textbook granularity trap**, with 4×8 partly advantaged by its coarser target space; aligned by fan-out (5×11 L3 $\approx 4 \times 8$ L3), the two shapes are much closer. Accordingly, SID-level recall should be used only to choose a coarse operating point or compare same-shape models, not as “recommendation quality” or the sole basis for cross-SID selection, which requires end-to-end evidence (within-bucket downstream / online A-B); for *early-stage* retrieval, coarse buckets are acceptable by design, and the real failure mode is being *too coarse* (fan-out in the hundreds), not insufficiently precise.

5.6 Pre-training Payoff (RQ3)

We next test whether self-supervised pre-training improves quality at fixed serving cost. AR uses a T5-style span-corruption objective and NAR a BERT-style masked-item objective before supervised fine-tuning. Table 9 isolates the effect head-to-head on the 12+12 model (the deepest-decoder $d=768$ configuration), holding architecture and supervised budget fixed.

Table 9: Pre-training payoff on 12+12 (4×8).

Metric	AR (ft)	AR (pt+ft)	Δ
HitRate@10	0.2773	0.2950	+6.4%
HitRate@100	0.5024	0.5228	+4.1%
NDCG@10	0.1728	0.1858	+7.5%
s/batch	0.273	0.274	≈ 0
	NAR (ft)	NAR (pt+ft)	Δ
HitRate@10	0.1308	0.1367	+4.5%
HitRate@100	0.3215	0.3309	+2.9%
NDCG@10	0.0737	0.0774	+5.0%
s/batch	0.313	0.308	≈ 0

At unchanged decode latency, HitRate@100 improves for both models: AR from 0.5024 to 0.5228 (+4.1%) and NAR from 0.3215 to 0.3309 (+2.9%). Convergence is also faster, reaching 0.459 after 2,000 fine-tuning steps and exceeding the from-scratch peak after 69% of the supervised budget. Because this gain carries over at no latency cost, we adopt pre-training throughout the deployable $d=768$ work-points of the following sweep (Table 10). The gain should be read as a warm-start effect, since the comparison matches supervised steps rather than total training compute.

5.7 Deployable Operating Point Efficiency (RQ3)

Generative retrieval follows a cost–quality ordering (ANN < NAR < AR), and within AR, encoder–decoder models are faster than single-stack models (1.81 vs. 2.41 s/batch) because they encode the context once; we therefore target shallow decoders.

Table 10 sweeps encoder depth, decoder depth, and width. We denote model depth as encoder+decoder layers; for example, 10+4 indicates a 10-layer encoder and a 4-layer decoder. d denotes the hidden dimension (e.g., $d = 768$). Decoding latency is measured in seconds per batch with a batch size of 256. Building on §5.6, the compact $d=768$ work-points (top block) are pre-trained (pt+ft), while the larger reference models (bottom block) are from-scratch

Table 10: Quality–latency sweep (4×8).

Model	Params	HitRate@10	HitRate@100	Precision@10	s/batch
8+1	89M	0.2562	0.4719	0.040	0.066
8+2	96M	0.2684	0.4884	0.042	0.076
8+3	103M	0.2744	0.4968	0.043	0.087
8+4	110M	0.2772	0.5003	0.044	0.101
10+1	98M	0.2572	0.4733	0.040	0.068
10+2	105M	0.2690	0.4888	0.042	0.080
10+3	112M	0.2760	0.4985	0.044	0.089
10+4	124M	0.2793	0.5028	0.044	0.103
12+1	112M	0.2612	0.4772	0.041	0.071
12+2	119M	0.2719	0.4923	0.043	0.081
12+3	126M	0.2776	0.5002	0.044	0.092
12+4	138M	0.2730	0.4943	0.043	0.105
<i>Larger models: deeper decoders and dimension</i>					
12+6	157M	0.2718	0.4944	0.043	0.137
12+12	214M	0.2773	0.5024	0.044	0.273
24+4 (d768)	250M	0.2698	0.4907	0.043	0.122
24+6 (d768)	268M	0.2747	0.4977	0.044	0.155
24+4 (d1024)	390M	0.2796	0.5034	0.045	0.150
24+6 (d1024)	424M	0.2840	0.5091	0.046	0.205

(ft). Decoder depth is the main latency driver: 1–2 layers require ~ 0.07 – 0.08 s/batch, 4 layers ~ 0.10 , 6 layers 0.14 – 0.16 , and 12 layers 0.27 , while encoder depth is cheaper because it is paid once per input. In the controlled $d=768$ grid, quality saturates at about 10 encoder layers, with no consistent gain from 10 to 12, though deeper or wider models can improve recall at higher cost.

The main operating point is **10+4** (124M, 0.5028 at 0.103s): a pre-trained shallow model matching the from-scratch 12+12 (0.5024) and 24+4 $d=1024$ (0.5034) at a fraction of their cost, since pre-training and a shallow decoder substitute for scale. For lower latency, 12+2 reaches 0.4923 at 0.081s. Even from-scratch 24+6 $d=1024$ (0.5091) decodes faster than 12+12 (0.205 vs. 0.273s): decoder cost is paid at every SID step, encoder cost once.

5.8 Ablations and Negative Results (RQ3)

Table 11 groups every controlled design choice into what *helps* and what does *not*. Four levers move quality: *switching route* (single-stack $\rightarrow$ enc-dec, +0.076 HitRate@100), the *trie constraint* (indispensable for NAR—it lifts recall 5.5 $\times$ and takes unconstrained legality from $\sim 3\%$ to 1.0; AR is far less dependent, +5–14%), *going*

Table 11: Controlled ablations (5×11 unless noted).

Design choice	Comparison	Result
Architecture	enc-dec vs single-stack	+0.076 HitRate@100 [big]
Trie constraint	constrained vs. free (NAR)	5.5 $\times$ recall [big]
Prefix re-decode	vs. prefix read (§5.4)	+9–34% [big]
Pre-training	pt+ft vs from-scratch	+3–4% HitRate@100 [moderate]
SID base	4×8 vs 5×11	operating point (§5.7)
Attention mask	prefix-LM vs causal	≈ 0 [low]
NAR capacity	24-layer vs 12-layer	≤ 0 , slower [low]
Codebook (RQ-VAE)	RQ-VAE vs base (NAR & AR)	≈ 0 [low]
Prefix-weighted CE	[3, 2, 1, 1, 1] vs uniform	+0.0008/lvl [low]
Beam width	50/100/200 (widen)	$\leq +0.006$ [low]
Target cap	32 vs 8	–5–7% (same-GT) [neg]

Summary. Architecture route, trie-constrained decoding, shallower re-decoding, and self-supervised pre-training improve retrieval quality.

shallower via truncated re-decode (§5.4, +9–34% coarse recall), and *self-supervised pre-training* (§5.6, +3–4% HitRate@100 at zero serving cost). The *SID base* is an operating point choice rather than a pure win: 4×8 deploys the shallow-decoder cheaply (§5.5, 5.7). Everything else is a null: the attention mask (prefix-LM), injecting the RQ-VAE codebook (reproduced null on both NAR and AR), tilting the CE toward coarse levels, widening the beam (50/100/200), raising the supervised target cap, and scaling NAR capacity (24-layer $\leq$ 12-layer and slower). Two nulls deserve a note because their naive version looks positive: increasing the target cap from 8 to 32 appears to add +13% recall, but a same-ground-truth re-evaluation shows it is a GT-set-size artifact (–5 to –7% aligned); and the codebook A/B is a clean cross-paradigm zero. The message: the levers are the *architecture paradigm*, the *trie*, going *shallower*, and *pre-training*—not capacity or decoding tricks.

5.9 AR-ANN Diversity Analysis (RQ4)

Having built a deployable model, we return to the 5×11 comparison substrate to ask how the generative route and ANN differ *by regime*—first per-input and by SID depth (here), then by signal density (§5.10)—and what combining them recovers. The per-level crossing of §5.3 suggests AR and ANN are complementary; Table 12 confirms it per-input. Their top-K lists barely overlap (mean overlap@100 6.5/100, Jaccard 0.037; 79.6% of retrieval examples share nothing in the top-10). More tellingly, even the *correctly retrieved positives* differ: at $K=100$ only 24% are found by both, and the union recalls $1.45\times$ the best single route; the ceiling widens toward the head ($1.64\times$ at $K=10$, $1.79\times$ at $K=1$). On diversity—a *diagnostic*, not quality—ANN is healthiest (Gini 0.615, top1pct-mass 0.134) while AR is most head-concentrated (Gini 0.898, top1pct 0.453). Read together: AR retrieves strongly but concentrates; ANN covers broadly; their positives are largely disjoint. This is a quantitative mandate for a hybrid retriever and bounds its ceiling.

Table 12: Per-input AR-vs-ANN overlap.

View	Metric	Value
(A) all retrievals	overlap@10 (mean; frac-zero)	0.30; 79.6%
	overlap@100 (mean; median)	6.5; 4
	Jaccard@100	0.037
(B) labelled-positive	both hit ($K=100$)	24.0%
	AR-only / ANN-only	45.0% / 31.0%
	union / best-single ($K=100$)	1.45 $\times$
	union / best-single ($K=1$)	1.79 $\times$

5.10 Head/Tail Segment Analysis (RQ4)

This analysis asks where Semantic-ID retrieval adds value relative to ANN as the available co-engagement signal changes. We segment the same 5×11 cold-start evaluation set by the number of observed ground-truth ad labels per input: **Head** (> 4 labels), **Torso** (3–4), and **Tail** (≤ 2). These groups contain 29%, 12%, and 59% of inputs, respectively. Both routes are evaluated on identical inputs with the same capped ground-truth denominator.

Table 13: Semantic-ID generative retrieval versus production ANN by input signal density.

Segment	Route	HitRate@100	Precision@10	NDCG@10
Head (> 4)	SID AR (5×11)	0.709	0.054	0.229
	ANN	0.545	0.037	0.152
	Δ (SID – ANN)	+16.4%	+1.7%	+7.7%
Torso (3–4)	SID AR (5×11)	0.562	0.033	0.162
	ANN	0.457	0.029	0.133
	Δ (SID – ANN)	+10.5%	+0.4%	+2.9%
Tail (≤ 2)	SID AR (5×11)	0.256	0.011	0.063
	ANN	0.232	0.012	0.070
	Δ (SID – ANN)	+2.4%	–0.1%	–0.7%

Absolute retrieval quality declines from Head to Tail for both routes (Table 13). This trend should be interpreted cautiously because inputs with more labels also provide more opportunities for a hit; here, label count is a proxy for observed co-engagement signal density rather than an independent causal measure of input difficulty. The more informative comparison is the within-segment difference between Semantic-ID AR and ANN. AR’s advantage is largest for signal-rich Head inputs (Δ HitRate@100=+16.4%; Δ NDCG@10=+7.7%), decreases in the Torso (+10.5% and +2.9%), and nearly vanishes in the Tail (+2.4% HitRate@100; –0.7% NDCG@10). Thus, generative retrieval does not primarily rescue sparse-signal inputs. Instead, it extracts the greatest advantage when rich co-engagement evidence is available; when the signal is sparse, both AR and ANN are constrained. This points to improving Tail-side signal quality or density, rather than model capacity alone, as the more promising path for further gains.

6 Conclusion

We recast early-stage ads retrieval as sequence generation over Semantic IDs and show that small language models (SLMs) can be both competitive and deployable. Representing items as short, coarse-to-fine code sequences lets a Transformer generate valid catalog identifiers under a trie constraint and reuse the standard LM recipe of self-supervised pre-training and supervised fine-tuning. Under fully cold-start evaluation, an encoder–decoder autoregressive SLM outperforms a strong production ANN baseline, and a controlled AR/NAR/ANN study shows that AR is the only route that simultaneously captures coarse and fine levels, while ANN remains a cheap and diverse complementary baseline.

The key advantage of Semantic IDs is that decode depth becomes a tunable *retrieval knob*: one full-depth model can be truncated at inference time (*train deep, infer shallow*), improving coarse recall while substantially reducing latency, something a fixed ANN index cannot offer. Self-supervised pre-training and deep-encoder/shallow-decoder architectures further push SLMs toward large-model quality at about half the parameters and serving cost. Finally, the largely disjoint positives of AR and ANN reveal a sizable hybrid recall ceiling, indicating that the most promising next step is a hybrid retriever that fuses generative and nearest-neighbor routes along the SID hierarchy.

References

- [1] Anirudhan Badrinath, Alex Yang, Kousik Rajesh, Prabhat Agarwal, Jaewon Yang, Haoyu Chen, Jiajing Xu, and Charles Rosenberg. 2025. OmniSage: Large Scale, Multi-Entity Heterogeneous Graph Representation Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 4261–4272. doi:10.1145/371896.3737253
- [2] Josh Beal, Eric Kim, Jinfeng Rao, Rex Wu, Dmitry Kislyuk, and Charles Rosenberg. 2026. PinCLIP: Large-scale Foundational Multimodal Representation at Pinterest. arXiv:2603.03544 [cs.CV] <https://arxiv.org/abs/2603.03544>
- [3] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*. 191–198.
- [4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2021. Autoregressive entity retrieval. In *International Conference on Learning Representations (ICLR)*.
- [5] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 4171–4186.
- [7] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. 2019. Unified language model pre-training for natural language understanding and generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32.
- [8] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. 299–315.
- [9] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-Predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6112–6121.
- [10] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time personalization using embeddings for search ranking at Airbnb. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 311–320.
- [11] Jiatao Gu, James Bradbury, Caiming Xiong, Victor O.K. Li, and Richard Socher. 2018. Non-autoregressive neural machine translation. In *International Conference on Learning Representations (ICLR)*.
- [12] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 3887–3896.
- [13] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*.
- [14] Min Hou, Le Wu, Yuxin Liao, Yonghui Yang, Zhen Zhang, Yu Wang, Changlong Zheng, Han Wu, and Richang Hong. 2026. A survey on generative recommendation: Data, model, and tasks. *AI Open* (2026).
- [15] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating long semantic IDs in parallel for recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.
- [16] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based retrieval in Facebook search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2553–2561.
- [17] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM International Conference on Information and Knowledge Management (CIKM)*. 2333–2338.
- [18] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011), 117–128.
- [19] Jian Jia, Jingtong Gao, Ben Xue, Junhao Wang, Qingpeng Cai, Quan Chen, Xiangyu Zhao, Peng Jiang, and Kun Gai. 2025. From principles to applications: A comprehensive survey of discrete tokenizers in generation, comprehension, recommendation, and information retrieval. *arXiv preprint arXiv:2502.12448* (2025).
- [20] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [21] Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. 2020. SpanBERT: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics (TACL)* 8 (2020), 64–77.
- [22] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*. 197–206.
- [23] Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.
- [24] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11523–11532.
- [25] Sunkyung Lee, Minjin Choi, and Jongwuk Lee. 2023. GLEN: Generative retrieval via lexical index learning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [26] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie, Xiangnan He, and Tat-Seng Chua. 2024. A survey of generative search and recommendation in the era of large language models. *arXiv preprint arXiv:2404.16924* (2024).
- [27] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2024. Learning to rank in generative retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [28] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [29] Gustavo Penha, Edoardo D’Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, and Hugues Bouchard. 2025. Semantic IDs for joint generative search and recommendation. In *Proceedings of the 19th ACM Conference on Recommender Systems (RecSys), Late-Breaking Results*.
- [30] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI technical report* (2019).
- [31] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research (JMLR)* 21, 140 (2020), 1–67.
- [32] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
- [33] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative retrieval with semantic tree-structured identifiers and contrastive learning. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 154–163.
- [34] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, et al. 2024. Better generalization with semantic IDs: A case study in ranking for recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys)*.
- [35] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450.
- [36] Qiaoyu Tang, Jiawei Chen, Zhuoqun Li, Bowen Yu, Yaojie Lu, Cheng Fu, Haiyang Yu, Hongyu Lin, Fei Huang, Ben He, Xianpei Han, Le Sun, and Yongbin Li. 2024. Self-retrieval: End-to-end information retrieval with one large language model. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [37] Yi Tay, Vinh Q Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 21831–21843.
- [38] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2017. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [40] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, et al. 2022. A neural corpus indexer for document retrieval. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 25600–25614.
- [41] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, Zhumin Chen, and Xin Xin. 2024. Content-based collaborative generation for recommender systems.

- In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*.
- [42] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, and Zhenhua Dong. 2024. EAGER: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.
 - [43] Liu Yang, Fabian Paischer, Kaveh Hassani, Jiacheng Li, Shuai Shao, Zhang Gabriel Li, Yun He, Xue Feng, Nima Noorshams, Sem Park, Bo Long, Robert D Nowak, Xiaoli Gao, and Hamid Eghbalzadeh. 2024. Unifying generative and dense retrieval for sequential recommendation. *arXiv preprint arXiv:2411.18814* (2024).
 - [44] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. 2019. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*. 269–277.
 - [45] Peiwen Yuan, Xinglin Wang, Shaoxiong Feng, Boyuan Pan, Yiwei Li, Heda Wang, Xupeng Miao, and Kan Li. 2024. Generative dense retrieval: Memory can be a burden. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
 - [46] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2024. Scalable and effective generative information retrieval. In *Proceedings of the ACM Web Conference (WWW)*.
 - [47] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *IEEE 40th International Conference on Data Engineering (ICDE)*.